Scaling Compute Is Not All You Need for Adversarial Robustness

Edoardo Debenedetti
ETH Zurich

Zishen Wan
Georgia Institute of Technology

Maksym Andriushchenko
EPFL

Vikash Sehwag
Princeton University

Kshitij Bhardwaj
Lawrence Livermore National Laboratory

Bhavya Kailkhura
Lawrence Livermore National Laboratory

arXiv:2312.13131v1 [cs.LG] 20 Dec 2023

***Abstract*—The last six years have witnessed significant progress in adversarially robust deep learning. As evidenced by the CIFAR-10 dataset category in RobustBench benchmark, the accuracy under ℓ_∞ adversarial perturbations improved from 44% in Madry et al. [25] to 71% in Peng et al. [29]. Although impressive, existing state-of-the-art is still far from satisfactory. It is further observed that best-performing models are often very large models adversarially trained by industrial labs with significant computational budgets. In this paper, we aim to understand: "how much longer can computing power drive adversarial robustness advances?" To answer this question, we derive *scaling laws for adversarial robustness* which can be extrapolated in the future to provide an estimate of how much cost we would need to pay to reach a desired level of robustness. We show that increasing the FLOPs needed for adversarial training does not bring as much advantage as it does for standard training in terms of performance improvements. Moreover, we find that some of the top-performing techniques are difficult to exactly reproduce, suggesting that they are not robust enough for minor changes in the training setup. Our analysis also uncovers potentially worthwhile directions to pursue in future research. Finally, we make our benchmarking framework (built on top of `timm` [41]) publicly available to facilitate future analysis in efficient robust deep learning.**

I. Introduction

Rapid advances in deep learning continue to improve performance on benchmarks across data modalities, achieving near-human, and in some cases even better than human performance. However, the challenge of achieving very high adversarial robustness remains elusive, despite substantial investmentment in robust learning mechanisms. While scaling of computational resources has become a de facto approach to drastically improve the performance of deep neural networks across numerous tasks [2, 3, 13, 20, 21, 35, 45], similar studies are nearly missing in the context of adversarial robustness. In this work, we take some first steps toward answering the question:

Is scaling compute a viable solution for achieving adversarial robustness?

Note that scaling of computational resources in deep learning has a direct increase in environmental impact. Thus we are further interested in a two-fold impact of scaling, i.e., whether continuing a "scaling compute" strategy would provide proportional improvement in adversarial robustness or would it have a disproportional environmental impact without much benefit to adversarial robustness.

To achieve this objective, we carry out a systematic empirical exploration by adversarially training a large number of models on CIFAR-10 dataset, the most commonly studied dataset on this task. Specifically, we vary several factors such as model size, adversarial loss, attack steps, use of synthetic data, etc. Next, we analyze the adversarial robustness of the resulting adversarially trained model zoo. Contrary to the intuitive notion that greater computational power (i.e., deeper and more complex models) should inherently improve a model's ability to withstand adversarial attacks, our research reveals an interesting phenomenon: *scaling up model sizes does not yield proportionate improvements in adversarial robustness*—setting adversarial training apart from standard training. This implies that exponentially more computational resources (and carbon emission) are required to achieve similar gains, making compute scaling an inefficient (if not infeasible) approach to solving this open problem.

We argue that the adversarial robustness community must think beyond the traditional paradigm of simply increasing compute. We must explore innovative avenues for addressing this important challenge, one that has clearly defied conventional approaches and has profound implications for the security and reliability of modern machine learning systems. In this work, we highlight the limitations of the current strategies and advocate for more creative and holistic approaches to achieving adversarial robustness, urging the field to break new ground in pursuit of robust and resilient AI systems.

II. Background and Related Work

A. Adversarial Robustness

a) Background and Threat Model. For a model $f : \mathcal{X} \to \mathbb{R}^C$, input $\boldsymbol{x} \in \mathcal{X}$, and label $y \in \{1, \dots, C\}$, we say that an adversarial perturbation $\boldsymbol{\delta} \in \Delta$ is *successful* if

$$\underset{c \in \{1,\dots,C\}}{\arg\max}\ f(\boldsymbol{x} + \boldsymbol{\delta})_c \neq y. \tag{1}$$

Following the previous established works [7, 25, 37], we assume that the attacker has *full knowledge* about f (i.e., a white-box threat model), and we measure adversarial robustness via *robust accuracy* for the ℓ_∞ norm, i.e., the fraction of points x on which the model f predicts the correct class y for all possible perturbations δ from $\Delta = \{\boldsymbol{\delta} \in \mathcal{X} \mid \|\boldsymbol{\delta}\|_\infty \leq \varepsilon\}$. We

note that robustness to small ℓ_∞-perturbations is of interest in different contexts: for robustness to unseen perturbations [19], better gradient-based interpretability [39], feature learning [32], generalization performance [42]. Many current robust training approaches are derived from *adversarial training* which is formulated as the following min-max robust optimization problem in Madry et al. [25]:

$$\min_f \mathbb{E}_{(\boldsymbol{x},y)\sim D}\big[\max_{\delta\in\Delta} \ell(f(\boldsymbol{x}+\boldsymbol{\delta}),y)\big], \tag{2}$$

where the inner maximization is typically approximated by multiple iterations of projected gradient descent (PGD). Importantly, since PGD has to be performed on every iteration of training, this significantly increases the computational cost of robust training.

b) Existing Robustness Benchmarks. There are multiple popular robustness benchmarks and libraries in the community. However, importantly, none of these benchmarks take into account the computational costs needed to train state-of-the-art robust models. Most libraries focus primarily on robustness *evaluation* and implement various white- and black-box adversarial attacks [28, 31, 27, 9, 26, 12, 23]. Other libraries focus on implementing defenses, most prominently Nicolae et al. [27] implement multiple defenses from the literature and Engstrom et al. [11] provides an implementation of adversarial training. As for benchmarks, one of the first benchmarks for different *attacks* is introduced in the works of Madry et al. [25] and Zhang et al. [46] whose models withstood many thorough robustness evaluations. RobustML (https://www.robust-ml.org/) was one of the early benchmarks that aimed at collecting robustness claims from the community for the most prominent defenses. Croce et al. [7] further introduced RobustBench, a popular adversarial robustness benchmark that includes a large repository of models and is based on AutoAttack [6], an ensemble of four hyperparameter-free attacks (white- and black-box) that has shown reliable performance over a large set of models from the literature. In our work, we follow Croce et al. [7] and also rely on AutoAttack for standardized measurement of ℓ_∞ adversarial robustness.

B. Neural Scaling Laws

a) Scaling in Deep Learning. The exploration into neural scaling laws, which describe how the performance of neural networks changes with varying model size, data, and computational budgets, has been consistently observed and validated across diverse domains like image classification [2, 45], language modeling [20, 35], and neural machine translation [3, 13]. Recent works have found that scaling up the data size, the model size, and the training schedule often leads to substantially improved performance. Kaplan et al. [20] elucidates the intricate relationship between model size, dataset size, and computational budget, spotlighting pivotal scaling laws foundational to the efficacy of expansive neural language models. This work also offers strategies to optimally allocate a predetermined compute budget. In a bid to empirically gauge the advantages of scaling, [1] introduces a robust methodology to reliably deduce scaling law parameters from learning curves. This methodology, rooted in the *extrapolation loss*, offers insights into how neural architecture variations influence scaling exponents. The work by [24] delves deeper into the pivotal role computation plays in AI evolution, presenting insights into potential inflection points where computational power might no longer yield substantial AI advancements.

b) Environmental Implications. The escalating computational requisites of deep learning invariably lead to surges in energy consumption. Strubell et al. [36] quantifies the approximate financial and environmental costs associated with training language models and proposes recommendations to reduce costs and improve equity in NLP research and practice. Thompson et al. [38] offers an introspective look at the computational underpinnings of DL models, suggesting that advancements in DL models are increasingly tethered to computational growth which poses economic, technical, and environmental challenges that beckon more efficient computational strategies. Schwartz et al. [33] emphasizes that the computational expenses of cutting-edge AI endeavors have grown by a factor of 300,000 in recent epochs, resulting in a very substantial carbon footprint. This underscores the imperative to prioritize efficiency as a cardinal metric, aligned with accuracy, to mitigate AI's environmental impact and bolster inclusivity. Echoing these sentiments, Henderson et al. [16] champions the cause of carbon accountability in AI, introducing a transparent and standardized reporting paradigm. However, similar studies are nearly missing in the context of adversarial robustness. Therefore, in this work, we pioneer an exploration into the neural scaling laws governing adversarial robustness.

III. Experimental Roadmap

A. Adversarial Training Setup

We run extensive experiments over several hyper-parameters, spanning design choices such as loss function, architecture, activation function, number of attack steps, and amount of data used. *Overall, we train 320 models*, requiring more than ten thousand GPU hours of training.

a) Robust Losses. The first axis we ablate is the adversarial training loss: we compare the original Adversarial Training (AT) loss (introduced earlier in Section II-A) to the TRADES [46] one. Briefly speaking, TRADES reformulates the original PGD loss as a regularization term to improve the trade-off between robust and standard accuracy. In order to do this, it has two loss components, weighted by a parameter β used to balance the trade-off. The two components are the loss on the original input $\boldsymbol{x}$, and the loss on the adversarial example found by maximizing the Kullback–Leibler divergence between the model output given the original input $\boldsymbol{x}$ and given the adversarial example $\boldsymbol{x}+\boldsymbol{\delta}$. Formally, the loss is:

$$\min_f \mathbb{E}_{(\boldsymbol{x},y)\sim D}\big[\ell(f(\boldsymbol{x}),y) + \beta \max_{\delta\in\Delta} \ell_{KL}(f(\boldsymbol{x}), f(\boldsymbol{x}+\boldsymbol{\delta}))\big]. \tag{3}$$

Interestingly, the TRADES and the AT losses require different amounts of compute: for TRADES, we need to run one forward pass in addition to the number of forward and backward passes needed to compute the adversarial example for the given training input, while for the PGD loss, we need to run only as many forward passes as the number of attack iterations. However, we note that the additional forward pass required by TRADES gets amortized when the number n of attack steps is larger. For example, for WideResNet-28-10 with 1 attack steps, TRADES increases the number of FLOPs for one epoch by $1.3\times$, while it increases by only $1.08\times$ when using 10 iterations.

b) Models. We test several sizes of WideResNet [44], with different depths and widths, i.e., 28-10, 34-10, 34-20, and 70-16. This is a common architecture in the adversarial training literature, and these are the most used size configurations for this architecture [7]. Studying these architecture sizes enables us to analyze models with a wide range of FLOPs per forward pass and parameters, from 10.5 GFLOPs and $36M$ parameters for WideResNet-28-10 to 77.6 GFLOPs and $266M$ parameters for WideResNet-70-16. Moreover, we also test two different activation functions: ReLU, used originally in WideResNet, and GELU [17]. Previous work observed that smooth activation functions improve robustness [43, 8]. However, also GELU comes at extra compute cost as it requires about $1.5\times$ the memory required by ReLU.

c) Attack Steps. We vary the number of attack iterations, training our models with 1, 2, 5, 7, and 10 iterations. Increasing attack iterations leads to stronger attacks. Obviously, training models with more steps require more FLOPs as each attack iteration requires one additional forward and one additional backward pass over the model.

d) Number of Training Epochs and Exponential Moving Average. We follow both the original training set-up from Zhang et al. [46], with 100 training epochs and no exponential moving average (EMA) of the model weights, and the set-up proposed by Gowal et al. [14]. The latter comprises training the model for 400 epochs with EMA of the model weights with a decay of 0.995[1]. This setup, not only requires $4\times$ the number of FLOPs for the overall training, but it also requires $2\times$ the memory as we need to keep both the weights being trained and those being averaged through EMA.

e) Synthetic Data. Finally, we test the contribution achieved by synthetic data for adversarial training. Previous work shows that extra unlabeled data provides a significant boost in performance [5] and that this holds also if the extra data is synthetically generated [15, 34, 40]. For this reason, we also test how much synthetic data helps with robustness in all the aforementioned scenarios. We use 1M synthetic CIFAR-10 images released by Gowal et al. [15]. However, we do not account for the synthetic data generation nor the generative model training in our FLOP count. This is mainly because there are many open-source pre-trained models that can be used for generating data, and, overall, the compute needed to generate the data is negligible compared to the overall training compute, and the synthetic data can be used across different training runs, hence the compute amortized.

[1]Note that we do not test exponential moving average with 100 epochs because, due to the exponential mechanism, 100 epochs are often not enough to benefit from EMA.

B. Metrics of Interest

1) Accuracy-Related Metrics. As we are interested in adversarial robustness, we report the AutoAttack accuracy of our models. AutoAttack [6], an ensemble of four different attacks, is the de-facto standard to benchmark the robustness of deep learning vision models. However, as running AutoAttack is computationally expensive, during training we evaluate the models using PGD-40 on a subset of the test samples, and apply early-stopping based on this metric. This also mitigates potential overfitting of the overall results to AutoAttack.

2) Efficiency-Related Metrics. To quantify the computational efficiency and environmental cost of adversarial robustness, we perform an analysis of the FLOPs, electricity consumption, and carbon emissions required to train a variety of popular robust models. A detailed description of our methodology is as follows:

a) FLOPs. To gauge model training efficiency in terms of floating-point operations per second (FLOPs), we employ the DeepSpeed Flops Profiler[2]. This measurement is acquired from all submodules of the model throughout the training process. We compute the average total number of FLOPs required for training the model (excluding the validation of the model performed after each epoch), hence accounting for number of epochs, model size, number of attack steps, etc. As DeepSpeed does not enable measuring the FLOPs required for the backward pass, we approximate those FLOPs to be $2\times$ the number of FLOPs needed for the forward pass [18].

b) Electricity Consumption. During model training, we repeatedly query the NVIDIA System Management Interface to sample GPU power consumption and get the average over all samples. By averaging these values and integrating them with the training time and the number of GPUs utilized, we estimated the overall power consumption in kilowatt-hours (kWh). Additionally, we incorporated the Power Usage Effectiveness (PUE) coefficient - a metric that encapsulates the ancillary energy expended to sustain the computational infrastructure, primarily cooling. With a reference PUE coefficient of 1.58 (the global average for data centers in 2018), we subsequently computed the cumulative electricity expenditure for training a model, using the U.S. average electricity rate of \$0.12/kWh.

c) Carbon Emission. To encapsulate the carbon footprint, we adopted the metric of *CO2-equivalents*. This metric facilitates a unified representation of the global warming potential intrinsic to diverse greenhouse gases, expressed as an equivalent quantity of CO_2 [10]. We harnessed the Machine Learning Emissions

[2]DeepSpeed Flops Profiler: https://www.deepspeed.ai/tutorials/flops-profiler/